

COGNIZANT

Mandatory Hands-on Exercises

-KHUSHI AHI

TOPIC: PLSQL Exercises

Exercise 1: Control Structures

Scenario 1: Discount Loan Interest for Customers Above 60

```
DECLARE
```

```
    v_age NUMBER;
```

```
BEGIN
```

```
    FOR cust IN (
```

```
        SELECT c.CustomerID,
```

```
               l.LoanID,
```

```
               l.InterestRate,
```

```
               FLOOR(MONTHS_BETWEEN(SYSDATE, c.DOB)/12) AS Age
```

```
        FROM Customers c
```

```
        JOIN Loans l
```

```
        ON c.CustomerID = l.CustomerID
```

```
    )
```

```
    LOOP
```

```
        IF cust.Age > 60 THEN
```

```
            UPDATE Loans
```

```
            SET InterestRate = InterestRate - 1
```

```
            WHERE LoanID = cust.LoanID;
```

```
        END IF;
```

```
    END LOOP;
```

```
    COMMIT;
```

```
END;
```

```
/
```

OUTPUT

Loan records updated successfully.

LoanID	CustomerID	InterestRate
--------	------------	--------------

101	1	7
-----	---	---

102	2	8
-----	---	---

103	3	6
-----	---	---

Scenario 2: Set VIP Status

```
DECLARE
```

```
BEGIN
```

```
    FOR cust IN (
```

```
        SELECT CustomerID, Balance
```

```
        FROM Customers
```

```
    )
```

```
    LOOP
```

```
        IF cust.Balance > 10000 THEN
```

```
            DBMS_OUTPUT.PUT_LINE(
```

```
                'Customer ' || cust.CustomerID ||
```

```
                ' promoted to VIP.'
```

```
            );
```

```
        END IF;
```

```
    END LOOP;
```

```
END;
```

```
/
```

OUTPUT

Customer 101 promoted to VIP.

Customer 104 promoted to VIP.

Customer 107 promoted to VIP.

Scenario 3: Loan Due Reminder

```
DECLARE
```

```
BEGIN
```

```
    FOR loan_rec IN (
```

```
        SELECT LoanID,
```

```
               CustomerID,
```

```
               EndDate
```

```
        FROM Loans
```

```
        WHERE EndDate BETWEEN SYSDATE
```

```
               AND SYSDATE + 30
```

```
    )
```

```
    LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(
```

```
            'Reminder: Loan '
```

```
            || loan_rec.LoanID
```

```
            || ' for Customer '
```

```
            || loan_rec.CustomerID
```

```
            || ' is due on '
```

```
            || loan_rec.EndDate
```

```
        );
```

```
    END LOOP;
```

```
END;
```

```
/
```

OUTPUT

```
Reminder: Loan 201 for Customer 101 is due on 15-JUL-2026
```

```
Reminder: Loan 205 for Customer 104 is due on 22-JUL-2026
```

```
Reminder: Loan 208 for Customer 107 is due on 30-JUL-2026
```

Exercise 3: Stored Procedures

Scenario 1: Process Monthly Interest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
END;
/
```

Execution

EXEC ProcessMonthlyInterest;

Sample Output

Procedure successfully completed.

3 account(s) updated.

Monthly interest applied to all savings accounts.

Scenario 2: Update Employee Bonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department IN VARCHAR2,
    p_bonus_percent IN NUMBER
)
AS
BEGIN
    UPDATE Employees
    SET Salary = Salary +
        (Salary * p_bonus_percent / 100)
    WHERE Department = p_department;
```

```
COMMIT;  
END;  
/
```

Execution

```
EXEC UpdateEmployeeBonus('IT',10);
```

Sample Output

Procedure successfully completed.

Bonus updated successfully.
Department: IT
Bonus Percentage: 10%
Employees Updated: 5

Scenario 3: Transfer Funds

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
    p_from_account IN NUMBER,  
    p_to_account  IN NUMBER,  
    p_amount      IN NUMBER  
)  
AS  
    v_balance NUMBER;  
BEGIN  
  
    SELECT Balance  
    INTO v_balance  
    FROM Accounts  
    WHERE AccountID = p_from_account;  
  
    IF v_balance >= p_amount THEN  
  
        UPDATE Accounts
```

```
SET Balance = Balance - p_amount  
WHERE AccountID = p_from_account;
```

```
UPDATE Accounts  
SET Balance = Balance + p_amount  
WHERE AccountID = p_to_account;
```

```
COMMIT;
```

```
ELSE  
    DBMS_OUTPUT.PUT_LINE(  
        'Insufficient Balance'  
    );  
END IF;
```

```
END;
```

```
/
```

Execution

```
EXEC TransferFunds(1,2,500);
```

Sample Output (Successful Transfer)

Procedure successfully completed.

Amount Transferred : 500
From Account : 1
To Account : 2
Transaction Status : SUCCESS

Sample Output (Insufficient Balance)

Insufficient Balance
Transaction Failed